

9/7/25

* Basic computer arch configuration

mem. : 4096×16

Flags : R, Z, N, etc.

ALU : 16 bit operations.

instruction : 16 bit.

Registers : AC, DR, AR

Bus structure : 16 bit.

* 8086 Architecture :

- 16-bit Microprocessor.

↳ on one cycle it can perform operation over 16-bit.

- Buses : Address bus, Data bus, control bus.

- size of address bus is 20 bit

so there are 2^{20} memory locations (1M)

- 16 bit data bus, 16 bit control bus.

- 64K I/O ports

- 14, 26 bit registers.

- 16 bit words.

- multiplexed address and data bus

- it can prefetch 6 instruction byte and queues them in order.

- it requires +5 volt VCC, GND, CLK

- clock frequency 5MHz to 8MHz.

- 40 pin dual

- BIU handles transaction of data and address for EU

- BIU performs fetching,

- Both BIU and EU operates asynchronously. to give for an overlapping instruction fetch and execution mechanism which is called pipelining.

⇒ Execution Unit.

- decodes instruction, generates control signals, executes in ALU

→ Registers

Ax	AH	AL	Accumulator
Bx	BH	BL	Base
Cx	CH	CL	count
Dx	DH	DL	Data
pointers	SP		Stack pointer
	BP		base pointer
Index	SI		source index
	DI		destination index.

← 8bit → ← 8bit →

← 16 bit →

- A, B, C, D are used for ^{general} data manipulation

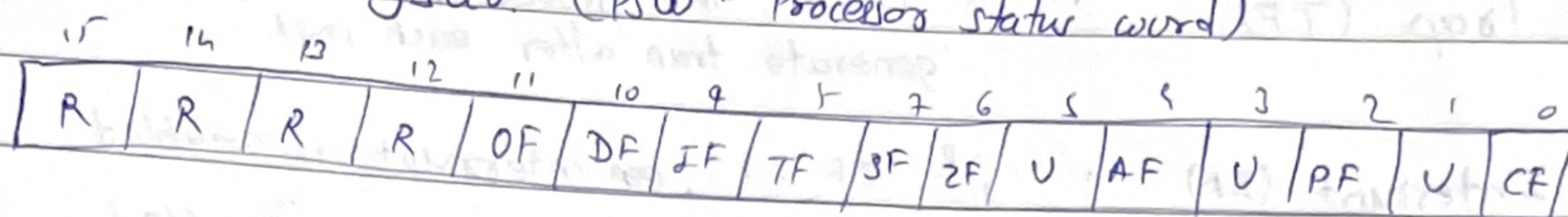
Ax : for arithmetic, logic, & transfer.

Bx : Base / address register.

Cx : for loop counter, shift, rotate

Dx : for multiplication, division or I/O op

Flag / Status Register (PSW - Processor status word)



R - Reserve

U - Undefined

OF - Overflow Flag

DF - Direction Flag

IF - Interrupt Instruction Flag

TF - Trap flag

SF - Sign Flag

ZF - Zero Flag

AF - Auxilliary Flag

PF - Parity Flag

CF - carry Flag

- Flag represent the status of bits after every operation (like add, sub, mul, etc.)

9 bits flag

7 unused flag bit

└ R

└ U

6 conditional flag

3 control flag

CF

└ PF

└ AF

└ ZF

└ SF

└ OF

Conditional Flags

Carry (CF) : holds carry for addition, borrow for subtraction.

Parity (PF) : parity of result 1 for even, 0 for odd

Auxilliary (AF) : if the whole 16 bit are not used (BCD op) then carry and/or borrow will be in AF

Zero (ZF) : if result is zero ZF = 1, else ZF = 0

sign (SF) : if result -ve SF = 1, else SF = 0

Overflow (OF) : if signed arithmetic generate carry OF = 1 else OF = 0

Control
Flag

Trap (TF) : if $TF = 1$, the processor execute 1 inst. and generate trap after each inst.

Interrupt (IF) : if $IF = 1$, interrupt is enabled
 $IF = 0$, interrupt is disabled

Direction (DF) : whether direction is forward/backward
 $DF = 0$, forward, $DF = 1$; backward

ex $AL = 7FH$ instruction $ADD AL, 1$ is executed

$$\begin{array}{r} AL \quad 7F \\ + \quad 01 \\ \hline AL = 80H \end{array}$$

$CF = 0$

$PF = 0$

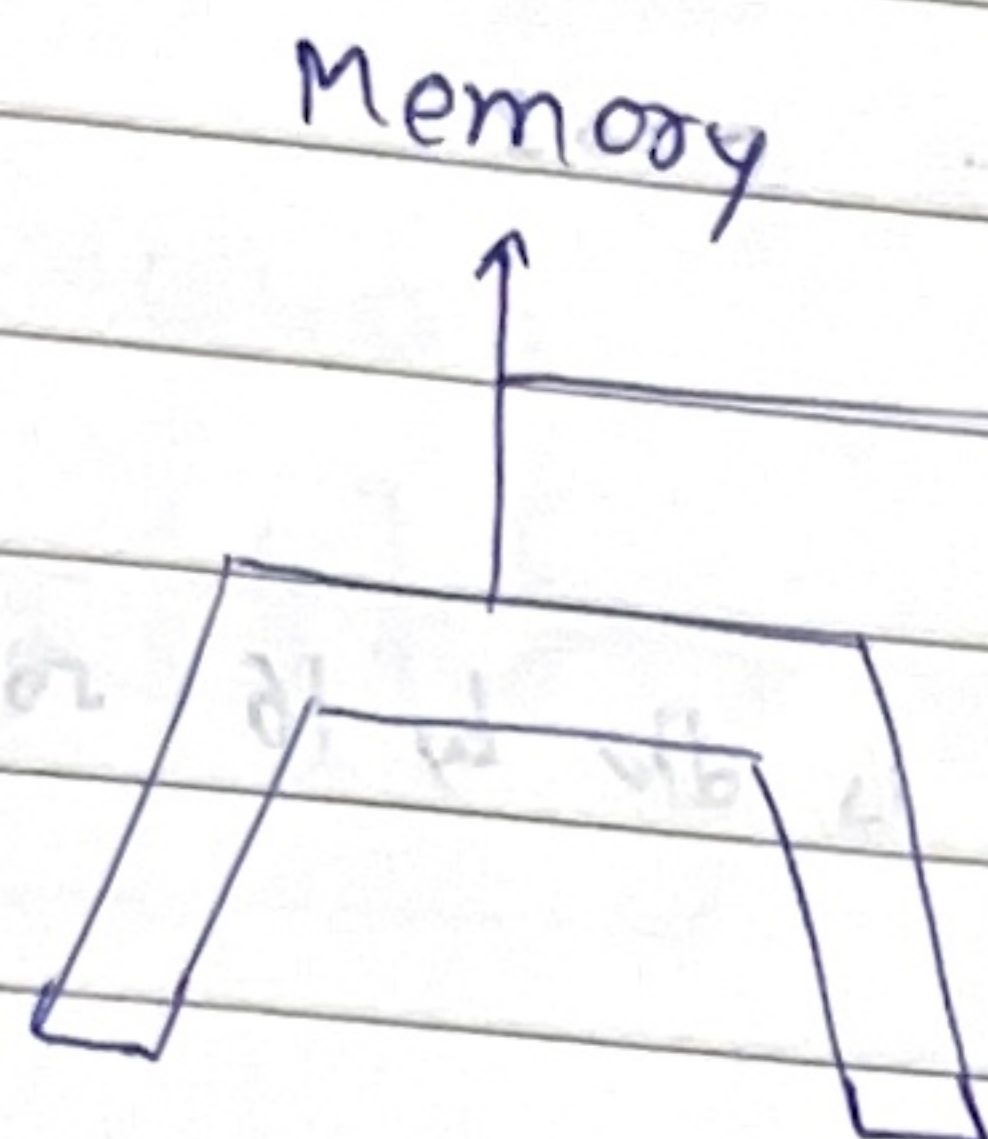
$AF = 1$

$ZF = 0$

sign = 0

$OF = 1$

⇒ Bus-Interface Unit



Instruction Pointer (IP)

- next instruction address is stored in IP
- instructions are of variable lengths
so after decoding only we can change IP/increment IP (by which)

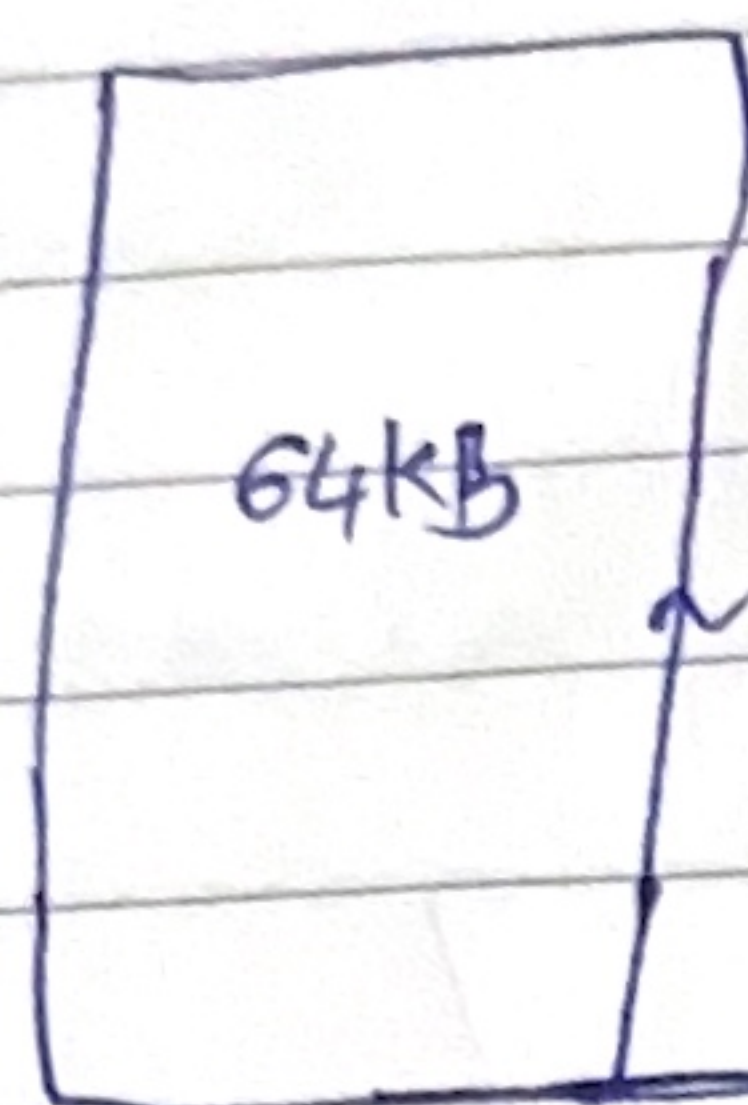
The queue

- 6 byte of instruction can be in fetch in queue

* Segment Memory.

- memory divided in segments like code segment, data segment, stack segment, extra segment.
- size maximum size of each segment can be 64KB.
- segment begin at any location which is divisible by 16.
↳ so the last digit in hexadecimal must be 0 like $xxxx0h$
- so we can store the starting address as 4 digit only because last digit is zero so it can be stored in segment register which is of 16 bits.
- There can be more than one segment can be defined but at a time only ~~se~~ one segment can be used
- 4 segments so 4 segment registers (16 bit) are used
- Segment can be overlapped or non-overlapped

offset/effective add. = 16 bit $\Rightarrow$ 0000h



64KB

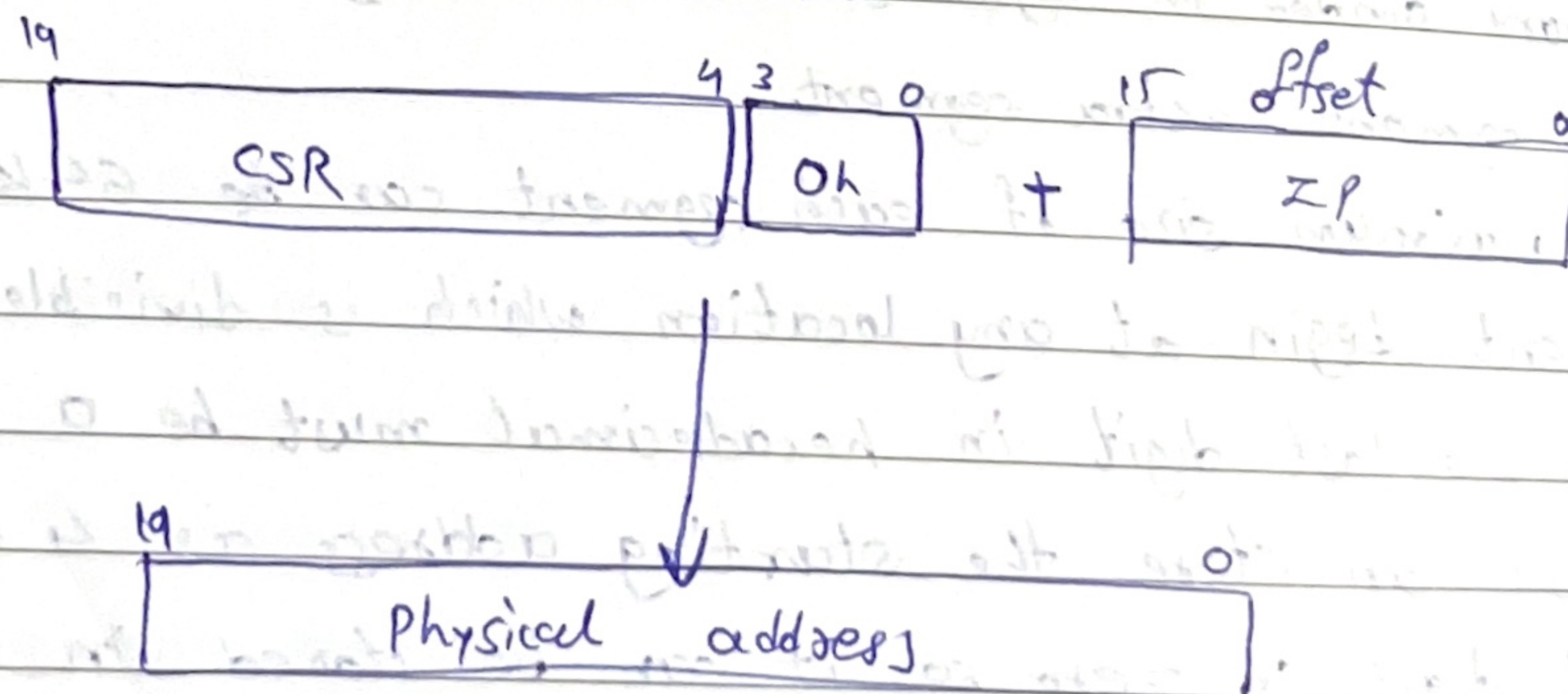
$\rightarrow$ 3rd element so offset = 0003

C2510h

seg. reg = 16 bit $\Rightarrow$ C251h ($\because$ because it is div by 16 so last is zero)

$$\begin{aligned} \text{so physical add} &= \text{seg. reg.} + \text{offset} \\ &= \text{C2510} + 0003 \\ &= \text{C2513} \checkmark \end{aligned}$$

- CS register stores starting address of code segment and IP (Instruction pointer) will always point to the offset.
- Similarly SS register and SP will also have offset.
- DS have starting add. of data segment and instruction will have the corresponding offset value.
- ES ~~have~~ is used for string operations. for offset in ES seg. it uses DI (Destination index).



CS : IP

SS : SP SS : BP

DS : BX DS : SI

ES : DI (for other than string operation)

ES : DI (for string operation).

16/7/26

eg.

CS = 76400H

ISR = 7040H

70400

+ 539C

phy 7579C

add

ESP: 3000H

30000

+ 8434

38434H

* Addressing Modes.

1.) Register addressing.

↳ segment registers cannot be used in this.

2.) Immediate

eg. MOV AX, 0123H

⇒

0123H AX

destination
operand

source operand

3.) Direct Addressing:

↳ operand address is given in the instruction.

eg.

data segment

offset	{	0000H	a	db	10H
add.	{	0001H	b	dw	123H

so,

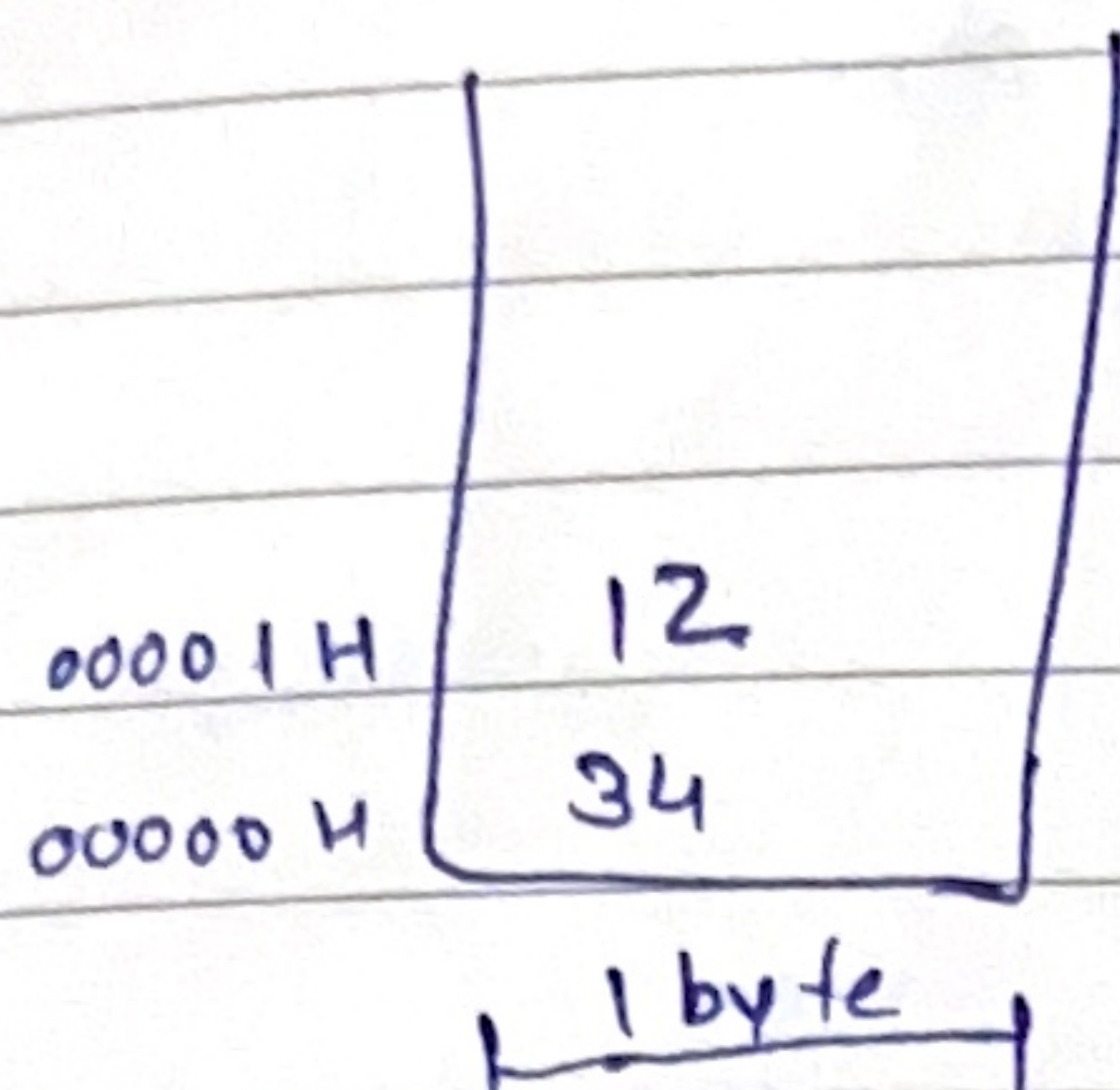
mov AL, a ⇒ mov AL, [0000]

is compiled to.

17/7/26

If data is larger than memory word then lower order values will be in lower addresses and higher order in higher address.

data = 1234H



eg. DS = 1000H

MOV AX, [20H]

add → 1000 0

20

AL ← 10020

AH ← 10021

Ex MOV, AX, [20H]

- Here address doesn't specify the length of data we can determine it with AX reg. only so in instruction one of the variable must be a general purpose register.

eg. ES = 2000H

inst. MOV ES:[1234H], AL

so AL is transferred/copied to 1234H offset in ES.

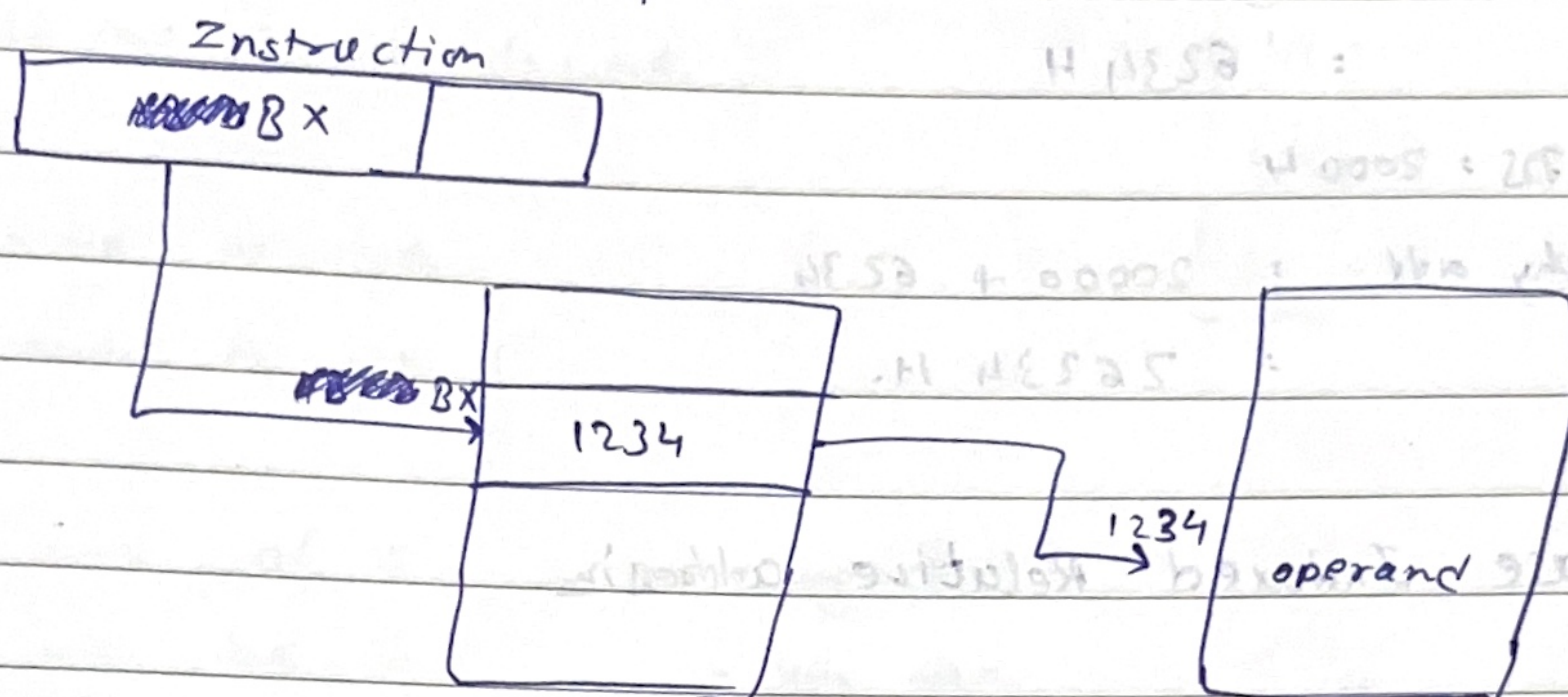
so 20000

+ 1234H

21234H in physical address.

4.) Indirect Addressing.

- In Instruction the address of Register is specified where the address of operand is present



eg. $10005 = 20$, $10005 = 12$, $10005 = 12$, $10005 = 12$

$\text{LEA } BX, a$

$\text{MOV } AL, [BX]$

5.) Base Relative Addressing.

eg.

$\text{MOV } AL, [BX + 34H]$

↓

$[1111 + 98 + 12]$ 16 bit or 8 bit

$7222 + 111$ (can be true or -ve)

for 8 bit $(-128 \text{ to } +127)$

$2000 - 32768 \text{ to } 32767$

$EA = [base] + displacement / \text{phy add } 1 = DS/SS + EA$

eg. $\text{MOV } AX, [BX] + 10$

eg. $(DS = 1000H) \quad BX = 0011H$

$\text{MOV } [BX + 777H], AX$

$[0111 + 777]$, AX

$[0888]$, AX

$DS + EA$

$10000 + 0888$

$10888, 10889$

6) Indexed Relative Addressing

eg. `mov DS, ARRAY(SI)`

$$\begin{aligned} EA &= \text{reg}(SI) + \text{ARRAY} \\ &= 8000 + 1234H \\ &= 6234H \end{aligned}$$

$$DS = 2000H$$

$$\begin{aligned} \text{phy add} &= 20000 + 6234 \\ &= 26234H. \end{aligned}$$

7) Base Indexed Relative Addressing

eg.

let $BX = 1000H$, $SI = 2000H$, $BEA = 1234H$, $DS = 1200H$
ARRAY :

$$\begin{aligned} EA &= \text{reg}(BX) + \text{reg}(SI) + \text{ARRAY} \\ &= 1000 + 2000 + 1234 \\ &= \end{aligned}$$

eg. let $DS = 1000H$, $SI = 3333H$, $BP = 2222H$

`mov AX, [SI + BP]`

or `AX, [5555]`

$$5555 + DS$$

$$\text{phy add} = 10000 + 5555$$

$$\text{phy} = 15555, 15556$$

add

AL

AH

`mov AX, [SI + BP + 1111H]`

$$1111 + 5555$$

`mov AX, [6666]`

$$\text{phy} = 10000 + 6666$$

$$= 16666, 16667$$

AL

AH

22/7/26

Data Segment

a db 11H st1 db 'ABCDEF'

b db 1234H

Data ends

Code Segment

assume cs:code, ds:data

start:

mov ax, data

mov ds, ax

mov al, a

- reg. direct

mov bx, b

- reg. direct

load effective address. (lea) si, st1

- reg. direct

mov cl, [si]

- indirect

mov ch, [si+3]

- index relative

mov bx, 1

- immediate

mov dl, [si+bx]

- base index

mov dl, [si][bx]

- same (base index)

mov dl, si[bx]

mov dl, bx[si]

mov dh, [si+bx+1] - base index relative

()

Here it can have bx, bl, bh, si, di

eg. BX = 0B39H, DS = 3745H, SS = 241AH, BP = 1234H, SI = 4235H,

DI = 3478H

mov cl, (BP) - ind.

mov AX, [5378H]

mov (DI+1778H), AX

mov AX, 5378H

mov (BX-1), BL

mov AX, (BX)(SI)

mov (DI, BP, 1111H), BX

1) $\text{mov } \text{CL}, [\text{BP}]$

$$\begin{array}{r} 241A0 \\ 1134 \\ \hline 253D4 \end{array} \rightarrow \text{CL}$$

3) $\text{mov } [\text{BX}-1], \text{BL}$

$$\begin{array}{r} 0B39 \\ 0B38 \leftarrow \text{BL (39)} \\ + 37420 \\ \hline 37FB8 \leftarrow \text{BX (37)} \end{array}$$

2) $\text{mov } (DI + 1778), \text{AX}$

$$\begin{array}{r} 3478 \\ 1778 \\ \hline 4BF0 \\ + 37480 \\ \hline 3C070 \leftarrow \text{AL} \\ 3C071 \leftarrow \text{AH} \end{array}$$

4) $\text{AX}, [\text{SI}][\text{BX}]$

Instruction Format.



D: Data TO/FROM REG, 0 = FROM 1 = TO

W: width of data transfer, 0 = Byte 1 = Word

REG: which register

R/M, MOD: for second operand

w=0	Register		code
	w=0	w=1	
AL	Ax	001	000
BL	Bx		011
CL	Cx	110	001
DL	Dx		010
AH	SP		100
BH	DI		111
CH	BP		101
DH	SI		110

Seg reg	code
CS	01
DS	111 1100
ES	00
SS	10

R/M	mod			w=0		w=1	
	00	01	10	11			
000	$[BX] + [SI]$	$[BX] + [SI] + d8$	$[BX] + [SI] + d16$	AL		Ax	
001	$[BX] + [DI]$	$[BX] + [DI] + d8$	$[BX] + [DI] + d16$	CL		Cx	
010	$[BP] + [SI]$	$[BP] + [SI] + d8$	$[BP] + [SI] + d16$	DL		Dx	
011	$[BP] + [DI]$	$[BP] + [DI] + d8$	$[BP] + [DI] + d16$	BL		Bx	
100	$[SI]$	$[SI] + d8$	$[SI] + d16$	AH		SP	
101	$[DI]$	$[DI] + d8$	$[DI] + d16$	CH		BP	
110	$[BP] + d16$	$[BP] + d8$	$[BP] + d16$	DH		SI	
111	$[BX]$	$[BX] + d8$	$[BX] + d16$	BH		DI	

eg. mov sp, bx

	D	W	MOD	REG	R/M	
100010	0	1	11	011	100	→ From
100010	1	1	11	100	011	→ To

- ↳ because both operands are registers instruction can be written in two ways.
- ↳ but if one is reg and other is memory then only one (from/to) will be used

2) mov cl, [bx]

100010 1 0 00 001 111

— dosbox / SASM / TASM

mount → cd

dir → ls

edit filename

tasm filename

tlink filename.obj → generates .exe file

td filename.exe → to debug.

debug. file.exe

↳ C - t

↳ G d ds : 0

BP	016 + (12)	06 + (12)	017
SI	016 + (10)	06 + (10)	015
DI	016 + (18)	06 + (18)	016
EBP	016 + (28)	06 + (28)	017

29/7/26

3) MOV 43H[SI], DH

100010 0 0 0 1 110 100 } 3 byte instruction.
 ↳ displacement: 0100 0011

4) MOV CX, [437AH]

100010 1 1 00 ~~001~~ 110 } 4 byte instruction
 ↳ address: 0111 1010 0100 0011
 7A 43

5) MOV CS:[BX], DL

100010 0 0 00 010 111 | 001 01 110
 2nd byte 3rd byte CS

1st byte↓
for segment override

10000

50000

10000

50000

Instruction execution timing.

		clock	memory ref
	ADD or SUB		
	reg to reg	3	0
read	- mem to reg	9 + EA	1
read + write	- reg to mem	16 + EA	2
	Imm to reg	4	0
read + write	- Imm to mem	17 + EA	2

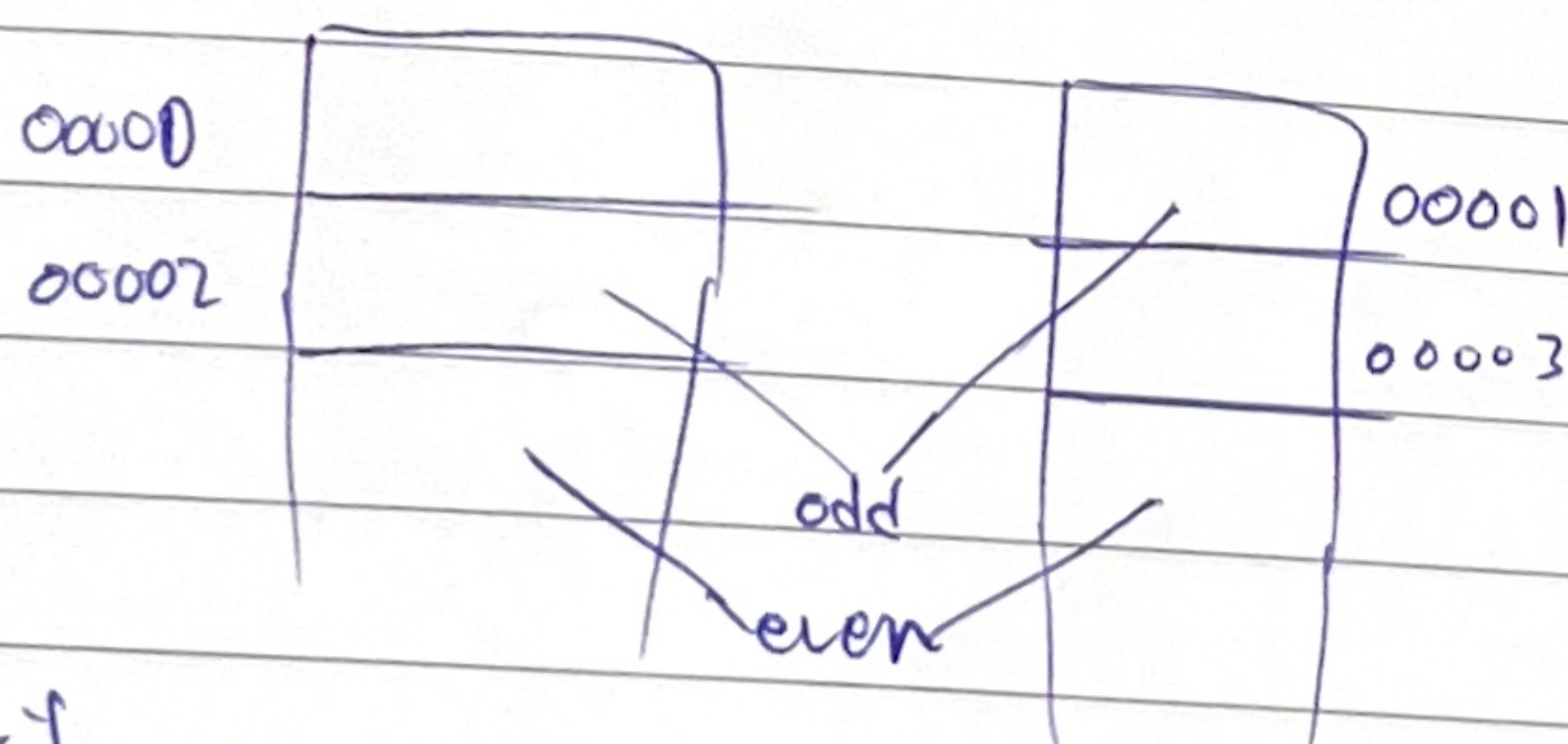
EA

Direct	6
Reg. Indirect	5
Reg relative	9
Base Index with BX/BP	7/8
Base index relative w BX/BP	11/12

clock freq : 5 MHz

$$T = \frac{1}{f} = 0.2 \mu\text{sec}$$

→ In 8086 memory is divided into ^{even} ~~odd~~ bank, ^{odd} ~~even~~ bank



- if ~~word~~ word starts from odd location, word is not in continuous allocated so additional 4 clock cycle is needed for each memory reference
- if word starts with even location, then whole word is stored sequentially so no extra clock cycle required